TOP 20 JAVASCRIPT INTERVIEW QUESTIONS

SEFA ALANUR

1. What is JavaScript?

JavaScript is a high-level, interpreted programming language used to create interactive and dynamic web content. It is a versatile, lightweight, and event-driven language that can run on both the client and server sides.

2. What are the data types supported by JavaScript?

JavaScript supports the following data types:

- **Primitive Types:** String, Number, BigInt, Boolean, Undefined, Null, Symbol.
- **Non-Primitive Types:** Object (including arrays, functions).

3. What is the difference between let, const, and var?

- **var:** Function-scoped, allows redeclaration, hoisted with undefined.
- **let:** Block-scoped, no redeclaration, hoisted but not initialized.
- **const:** Block-scoped, no redeclaration, and must be initialized during declaration.

4. Explain how == and === differ.

- == checks for value equality with type coercion.
- === checks for strict equality without type coercion.

5. What is a closure?

A closure is a function that retains access to its outer scope variables even after the outer function has executed.

```
function outer() {
  var outerVar = "Hello";

  function inner() {
    console.log(outerVar);
  }

  return inner;
}

var closureFn = outer();
closureFn(); // Output: Hello
```

6. What is hoisting?

Hoisting is JavaScript's behavior of moving variable and function declarations to the top of their scope during compilation.

```
var greeting = "Hello, world!";

function sayHello() {
  console.log(greeting);
}

sayHello();
```

7. Explain the concept of this in JavaScript.

this refers to the context in which a function is executed. Its value depends on how the function is called.

```
const obj = {
  name: "Siva",
  greet() {
    console.log(this.name);
  },
};
obj.greet(); // Siva
```

8. What are JavaScript prototypes?

Prototypes allow objects to inherit properties and methods from other objects.

```
function Person(name) {
  this.name = name;
}
Person.prototype.greet = function () {
  return `Hello, ${this.name}`;
};
const person = new Person("Siva");
console.log(person.greet()); // Hello, Siva
```

9. What is the difference between null and undefined?

- **null:** A deliberate non-value.
- **undefined:** A variable declared but not assigned a value.

10. How does JavaScript handle asynchronous operations?

JavaScript uses the event loop with callbacks, promises, and async/await for asynchronous operations.

11. What is a promise?

A promise represents the eventual completion or failure of an asynchronous operation.

```
const promise = new Promise((resolve, reject) => {
  setTimeout(() => resolve("Success"), 1000);
});
promise.then(console.log); // Success
```

12. What are async/await functions?

async/await allows writing asynchronous code that looks synchronous.

```
async function fetchData() {
  const data = await fetch("https://api.example.com");
  return data.json();
}
```

13. What is event delegation in Javascript?

Event delegation is a technique where you attach a single event listener to a parent element, and that event listener handles events occurring on its child elements. It helps optimize performance and reduce memory consumption.

```
// HTML:
<ul id="list">
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>

// JavaScript:
document.getElementById("list").addEventListener
("click", function(event) {
  if (event.target.nodeName === "LI") {
    console.log(event.target.textContent);
  }
});
```

14. What Is Memoization in JavaScript?

Memoization is a technique that can be used to improve the performance of JavaScript code. Memoization works by storing the results of expensive calculations in a cache. This allows the JavaScript code to avoid re-performing the expensive calculations if the same input is provided again.

For example, the following code calculates the factorial of a number. The factorial of a number is the product of all the positive integers from one to the number.

```
function factorial(n) {
  if (n === 0) {
    return 1;
  } else {
    return n * factorial(n - 1);
  }
}
```

15. What is the event loop?

The event loop processes tasks from the queue and stack for asynchronous operations.

```
console.log("Start");
setTimeout(() => console.log("Timeout"), 0);
console.log("End");
```


16. How can you prevent a function from being called multiple times?

You can use a debounce function.

```
function debounce(func, delay) {
  let timeout;
  return (...args) => {
    clearTimeout(timeout);
    timeout = setTimeout(() => func(...args), delay);
  };
}
```

17. What Is the Purpose of the bind() Method in JavaScript?

The bind() method is used to create a new function with a specified this value and an initial set of arguments. It allows you to set the context of a function permanently.

```
const person = {
  name: "John",
  greet: function() {
    console.log("Hello, " + this.name);
  }
};

const greetFn = person.greet;
const boundFn = greetFn.bind(person);
boundFn(); // Output: Hello, John
```

18. What Is the Purpose of the reduce() Function in JavaScript?

The reduce() function is used to reduce an array to a single value by applying a function to each element and accumulating the result.

```
const numbers = [1, 2, 3, 4, 5];
const sum = numbers.reduce(function(acc, num) {
  return acc + num;
}, 0);
console.log(sum); // Output: 15
```


19. Explain the concept of "event bubbling" and "event capturing".

- **Event Bubbling:** Events propagate from the target element to the parent elements.
- **Event Capturing:** Events propagate from the parent elements to the target element.

20. What Is a Generator Function in JavaScript?

A generator function is a special type of function that can be paused and resumed during its execution. It allows generating a sequence of values over time, using the yield keyword.

```
function* generateNumbers() {
  yield 1;
  yield 2;
  yield 3;
}

const generator = generateNumbers();
console.log(generator.next().value); // Output: 1
console.log(generator.next().value); // Output: 2
console.log(generator.next().value); // Output: 3
```